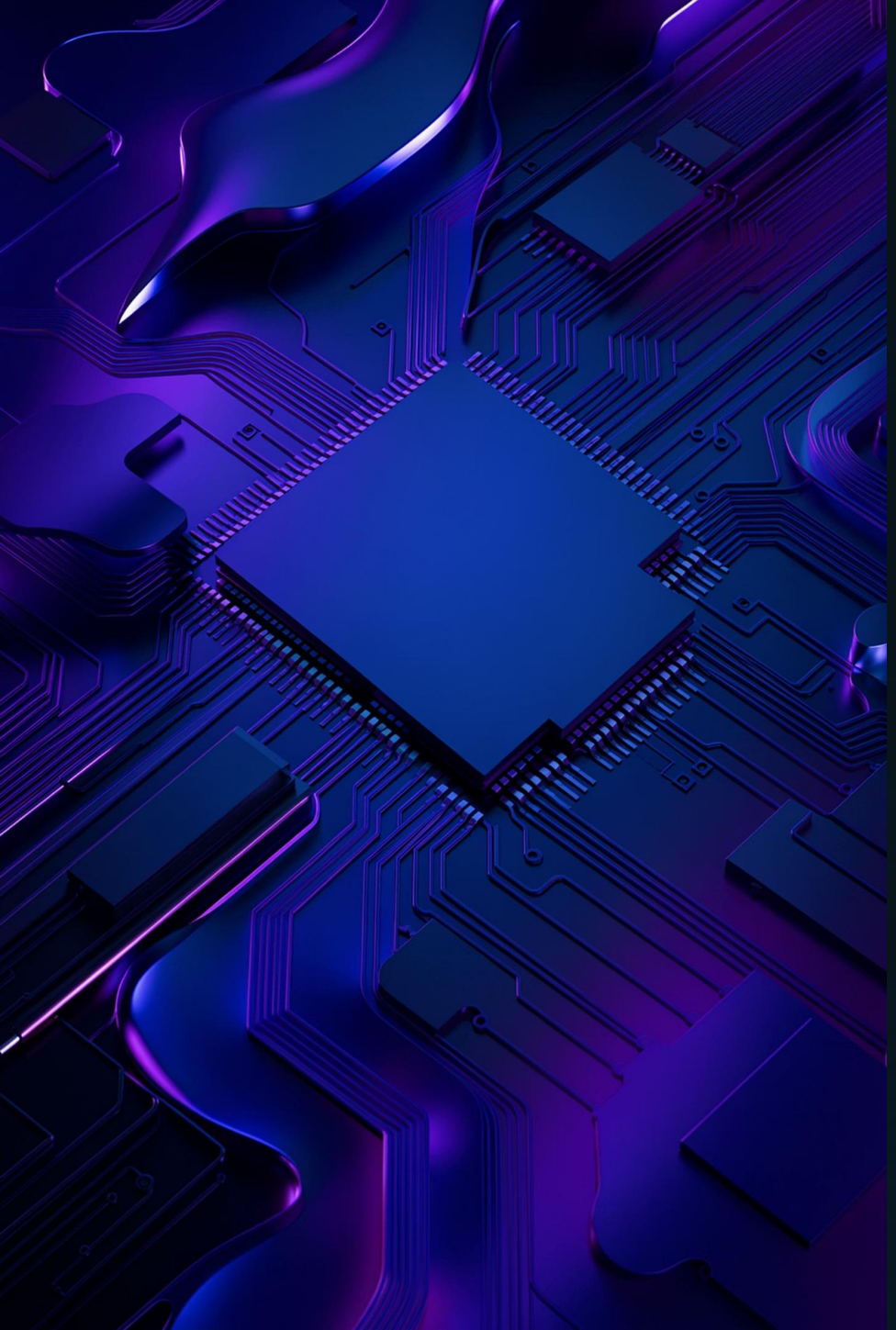


University of Sheffield

Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS/Non-CMOS Memories

Yuqi Guo — 230184273

Supervisor: Mr. Neil Powell | Second Marker: Dr. Mohammad Eissa

Project Overview

01

Motivation and Problem

Hybrid memories face spatially correlated cluster faults

02

Project Goal and Objectives

FPGA-based hardware acceleration platform design

03

Methodology and Experimental Setup

Two-phase evaluation framework

04

Main Innovation

Probabilistic Fault Injection Engine

05

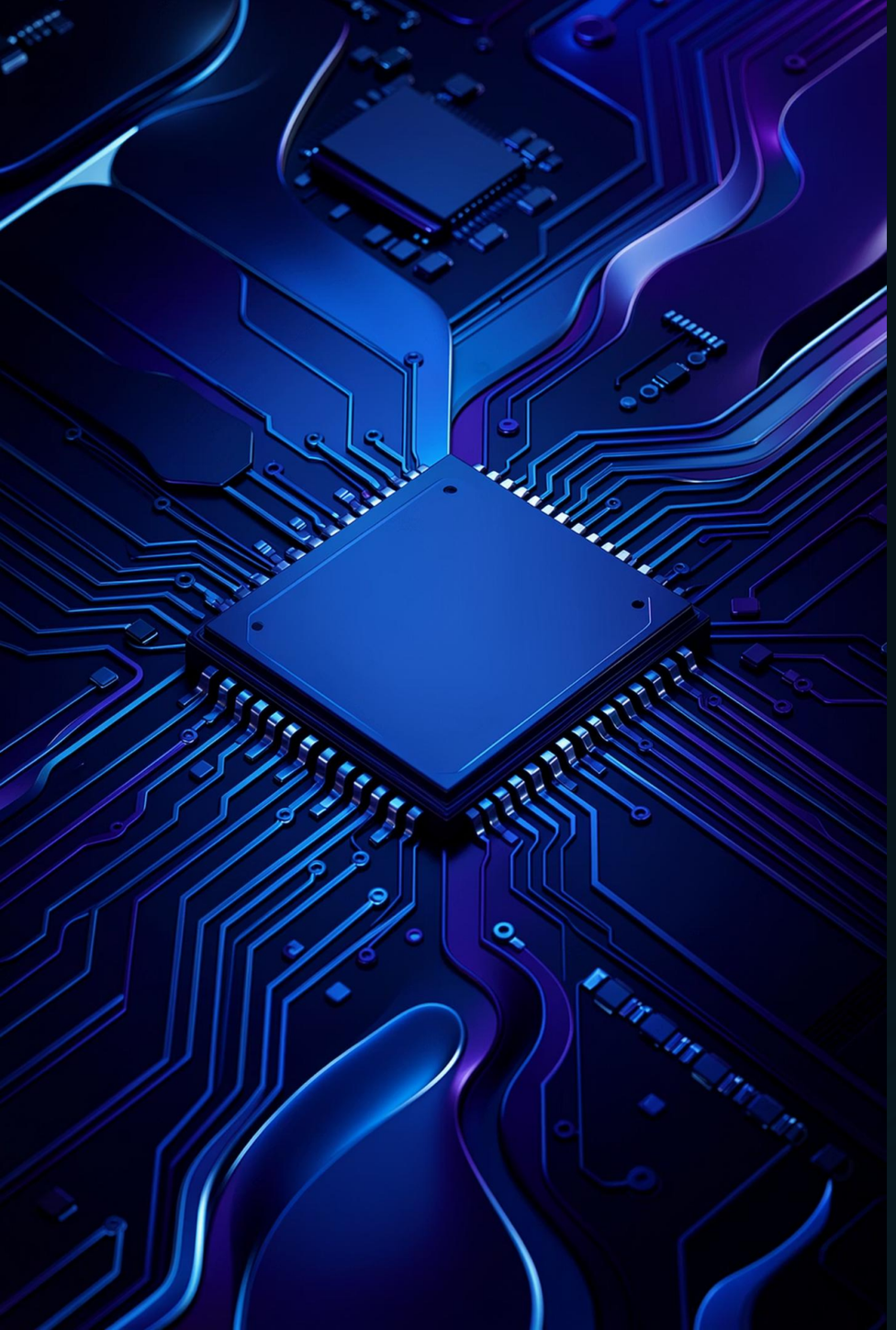
Results

Random single-bit and cluster burst analysis

06

Contributions and Future Work

Extensible architecture and findings



University of Sheffield

Background and Research Context

Why This Project Matters

Hybrid CMOS/non-CMOS memories offer high density and low cost, but they are vulnerable to **spatially correlated cluster faults**. Traditional ECC schemes are often evaluated for independent random errors, not realistic burst faults. Previous studies mainly relied on simulation; **hardware-validated comparison** was still missing.

Core Research Question

Which ECC design gives the best trade-off between fault tolerance, latency, area, power, and storage efficiency on real FPGA hardware?

Project Goal and Objectives

Project Goal

To design and implement an **FPGA-based hardware acceleration platform** for fair benchmarking of multiple ECC algorithms under realistic cluster-fault conditions.

Implementation Configurations

- 2NRM-RRNS Parallel
- 2NRM-RRNS Serial
- 3NRM-RRNS
- C-RRNS-MLD
- C-RRNS-MRC
- RS(12,4)

Fault Tolerance

Burst error correction capability

Decoding Latency

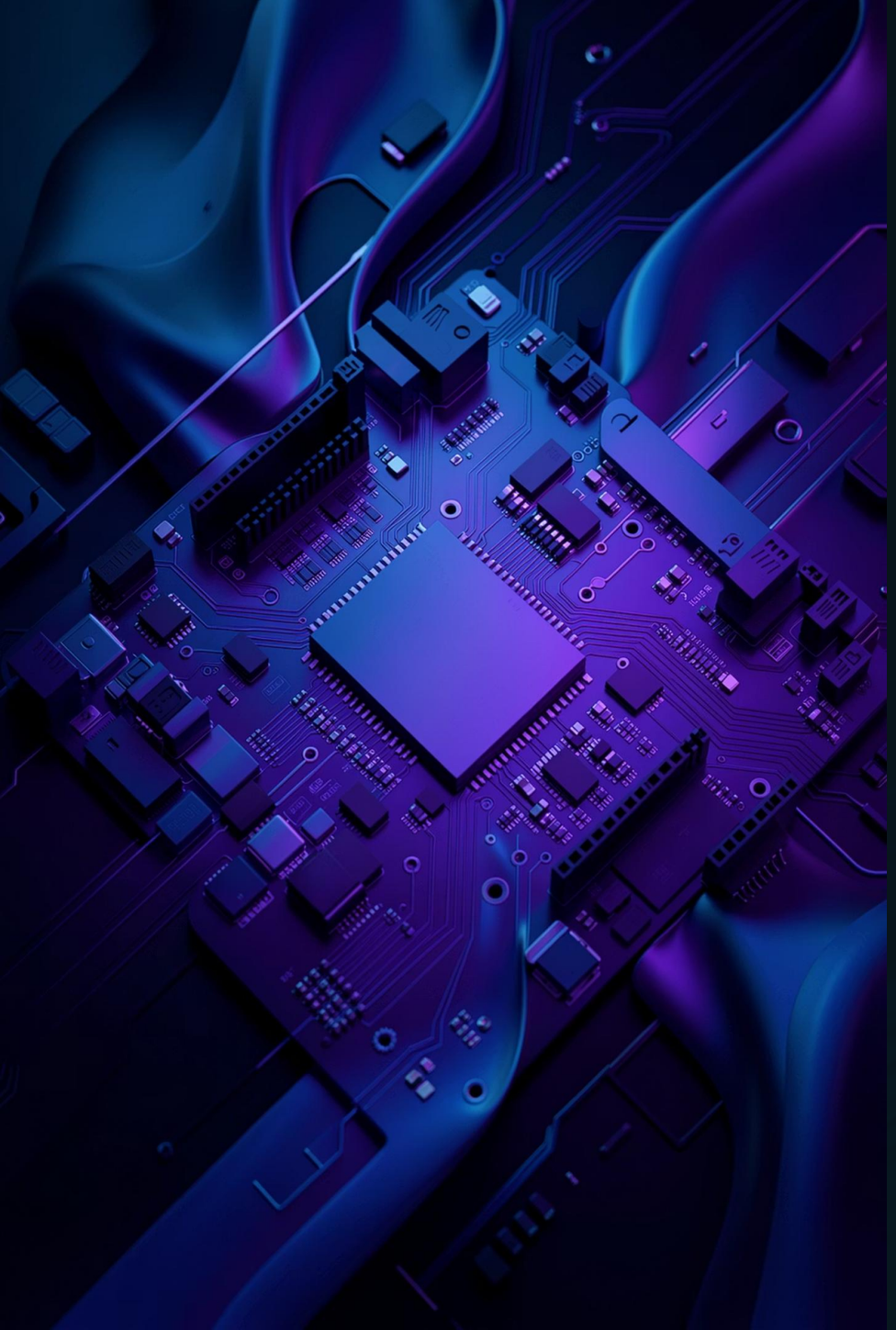
Processing time per frame

FPGA Resources

Area and power consumption

Storage Efficiency

Code rate and overhead



Methodology and Experimental Setup

Phase 1: MATLAB Simulation

Theoretical baseline analysis

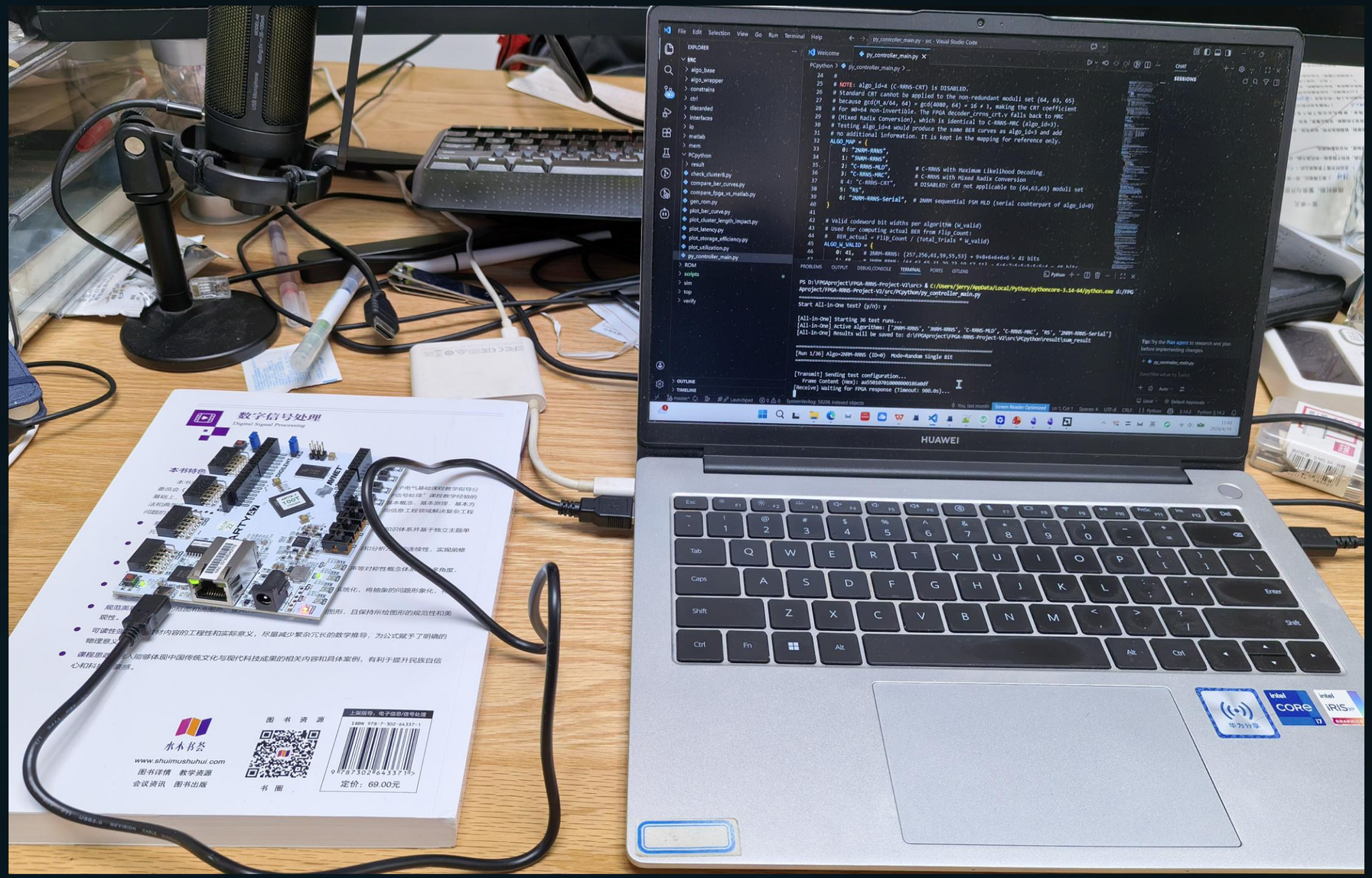
Phase 2: FPGA Implementation

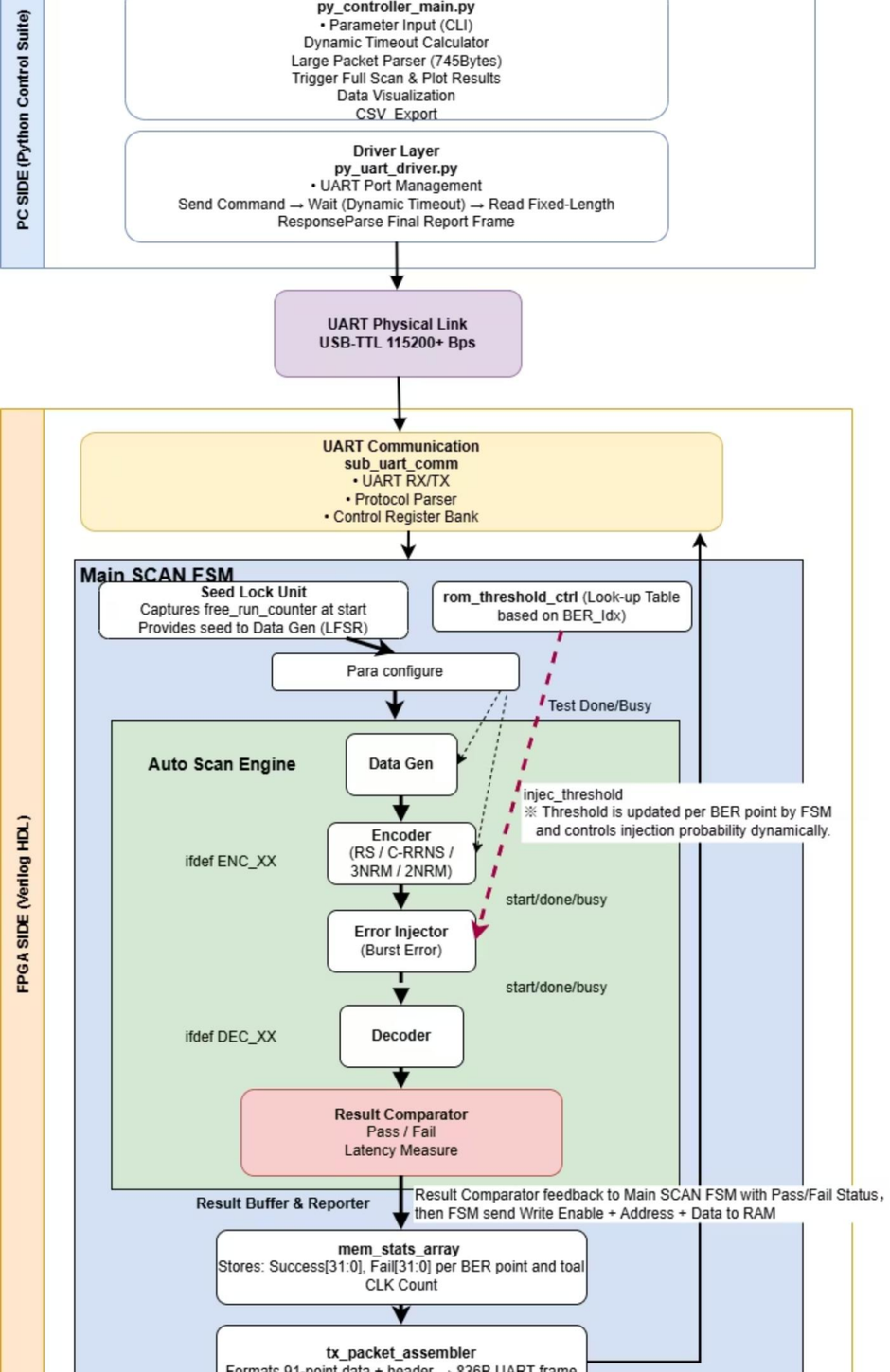
Hardware validation

Test Settings

- Device: Xilinx Artix-7 xc7a100t (Arty A7-100T)
- System Clock: 50 MHz
- Fault Injection Modes: Random single-bit and cluster burst (length 1-15)

FPGA test platform under working





FPGA Platform Architecture

Key Idea

The PC controls configuration and visualization, while the FPGA autonomously performs the full BER sweep and uploads consolidated results through UART. This architecture enables **scalable, hardware-validated testing** with minimal host intervention.

Main Innovation: Probabilistic Fault Injection Engine

1

32-bit Galois LFSR
Pseudo-random source for fault generation

2

Dual Injection Modes
Random single-bit and cluster burst support

3

Threshold Control
BER controlled by dynamic comparison

4

Precomputed Masks
Boundary-safe burst patterns in ROM

Why It Matters

This makes the platform **resource-efficient, reproducible, and reusable** for benchmarking different ECC algorithms.

Why the Test Platform Is Strong

Fair Comparison

All algorithms share the same test infrastructure and clock conditions

Hardware Validated

Results measured on Xilinx Artix-7 FPGA, not simulation alone

Low Overhead

Probabilistic injector uses only two BRAMs

Scalable Experiments

101 BER points from 0% to 10%, 100,000 samples per point

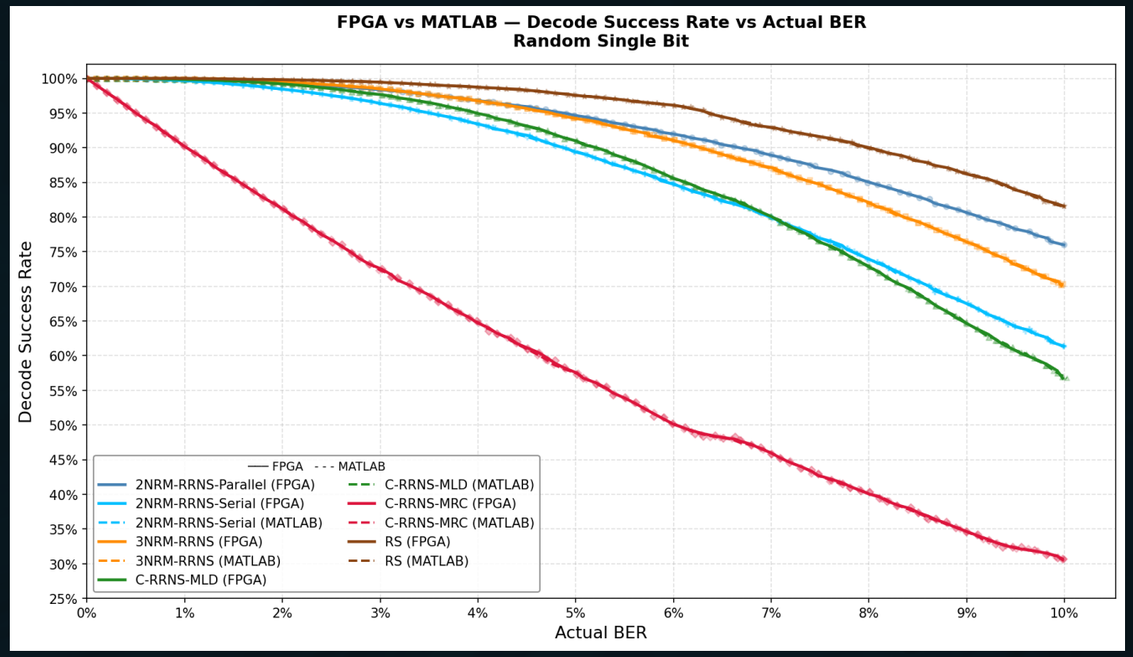
Extensible Architecture

New ECCs added with wrapper interface

Cross-Validated

FPGA curves closely match MATLAB results

Random Single-Bit Results



1000000 samples each BER

Main Observations

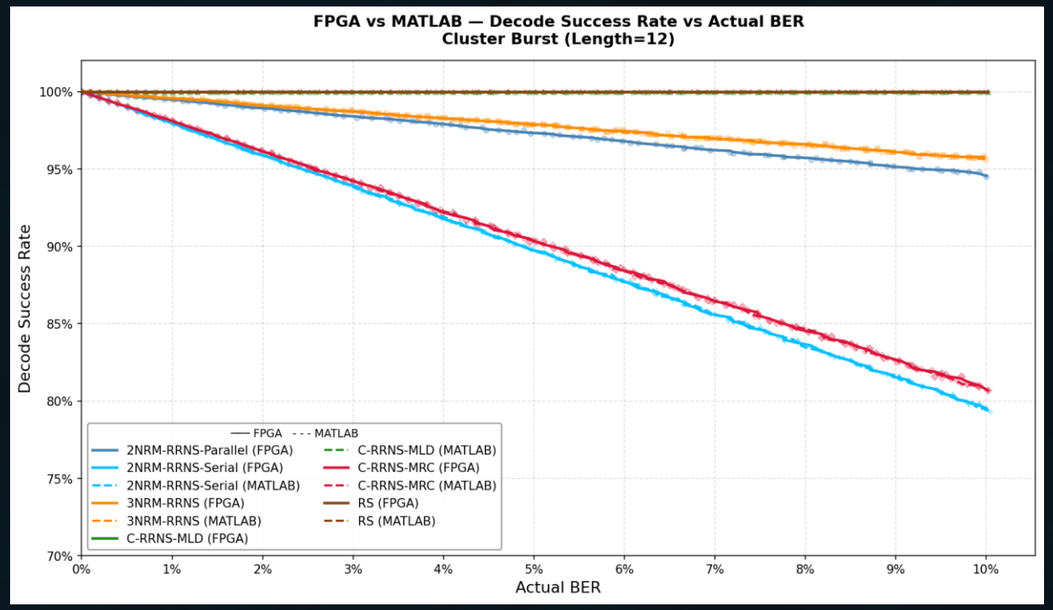
Model Agreement

FPGA and MATLAB curves show close agreement across all BER points

Differential Performance

Correcting ECCs remain strong at low BER, while C-RRNS-MRC drops fastest at higher error rates

Cluster Burst Results at Representative Length L=12



Key Findings

Top Performers

C-RRNS-MLD and **RS(12,4)** show the strongest burst resilience

Model Dependence

Burst-fault ranking differs from random-bit testing, showing why a realistic fault model is necessary

Quantitative Comparison of Key Results

A comprehensive hardware evaluation of error-correcting codes for cluster fault tolerance in FPGA memory systems.

Most Reliable

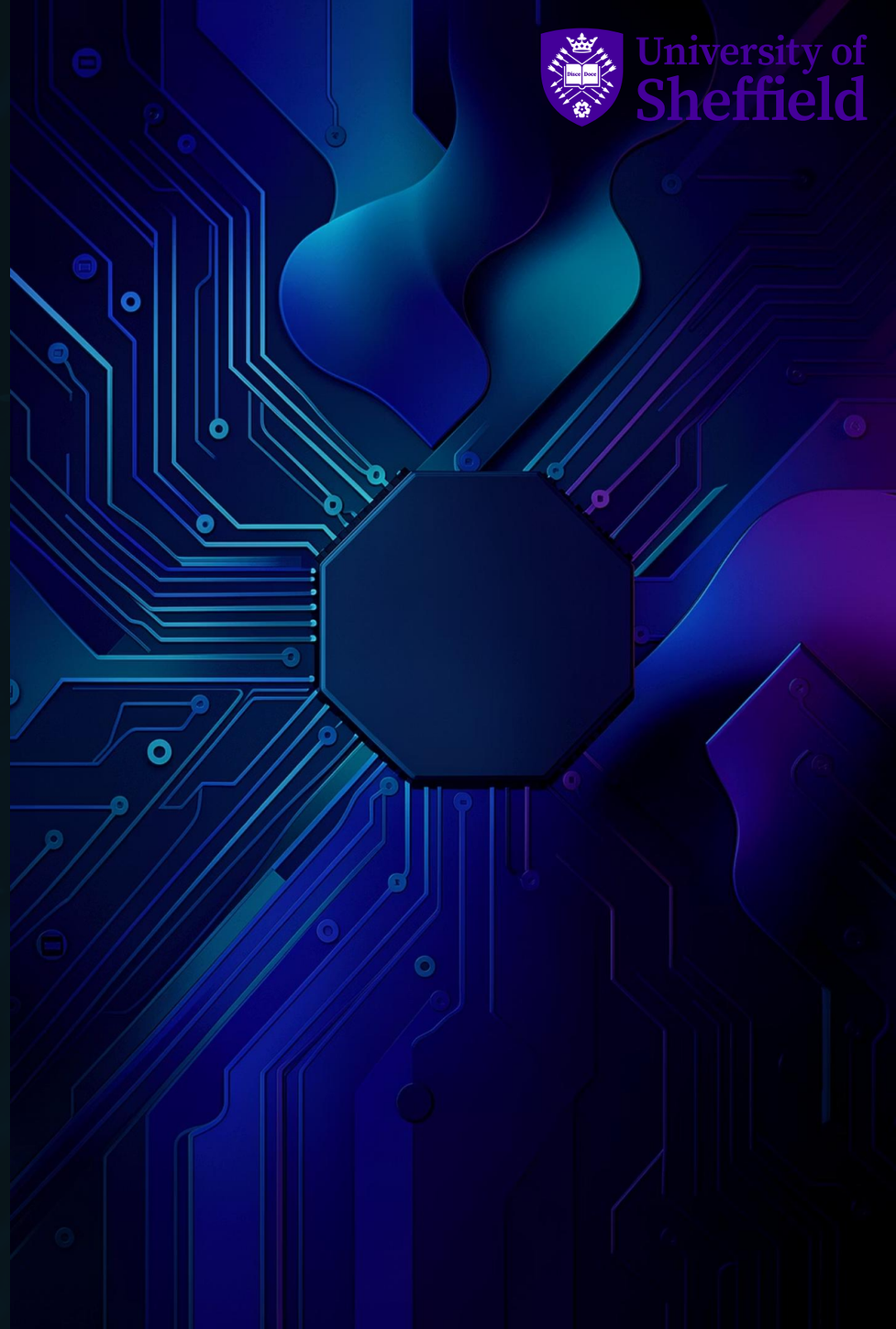
C-RRNS-MLD provides the strongest burst-fault tolerance in the explored design space

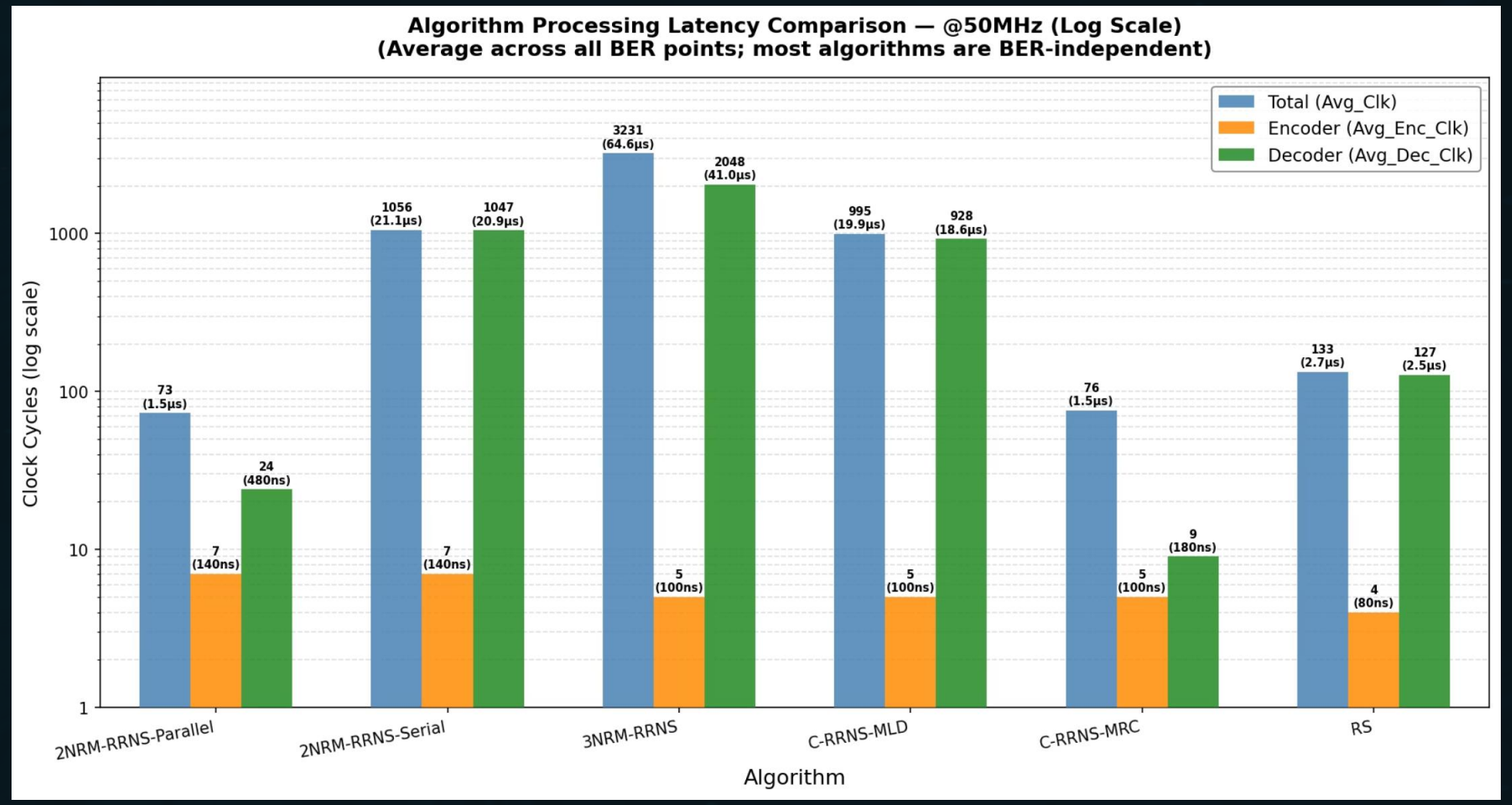
Fastest

2NRM-RRNS Parallel decoder achieves the lowest decode latency through parallel implementation

Most Compact

2NRM-RRNS offers the best storage efficiency for overhead-sensitive systems



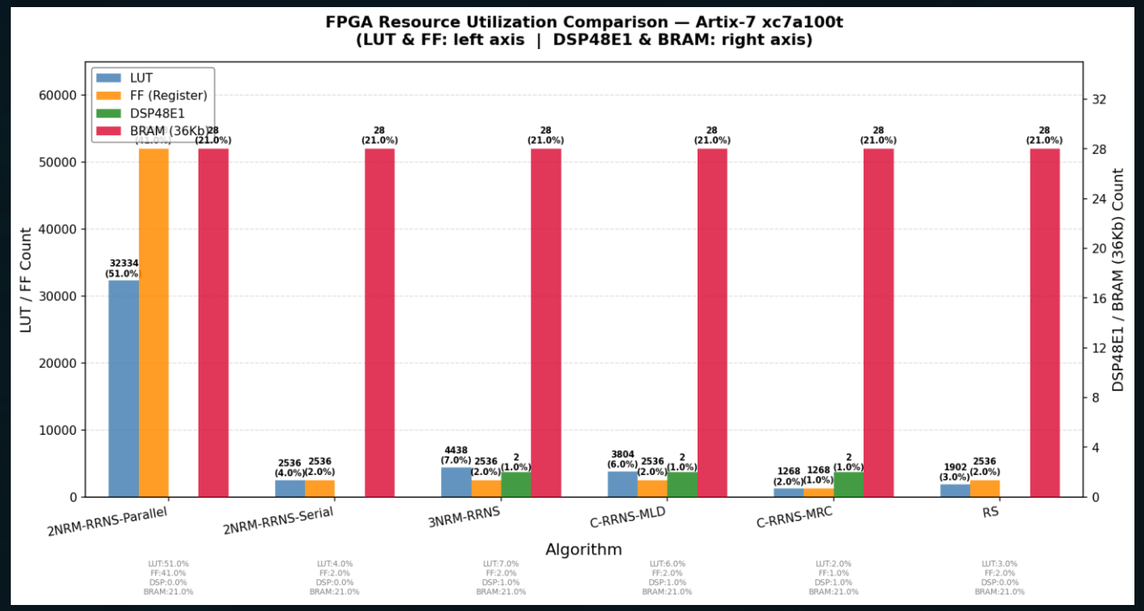


Latency Comparison

Performance Analysis

- The **2NRM-RRNS Parallel** decoder is the fastest correcting design
- Parallel implementation achieves major acceleration over serial version
- Results expose the **resource-latency trade-off** of parallel MLD architectures

Technical Note: The parallel implementation demonstrates a clear performance-speed trade-off, achieving faster correction at the cost of increased LUT and FF utilization.



Resource and Storage Comparison

Key Trade-offs

2NRM-Parallel

Achieves high speed at highest LUT/FF and power cost

2NRM-RRNS

Most storage-efficient option for overhead-sensitive designs

C-RRNS-MLD

Provides strongest protection with balanced resource usage

Application scenario recommendations

Scenario	Primary Constraint	Recommended
High-reliability systems (aerospace, medical)	Fault tolerance	C-RRNS MLD or RS
Storage-constrained devices (edge computing, IoT)	Codeword overhead	2NRM-RRNS (Parallel)
Latency-sensitive systems (real-time memory controllers)	Processing speed	RS(12,4) or 2NRM-RRNS (Parallel)
Resource constrained FPGAs (low-cost devices)	LUT utilization	2NRM-RRNS (Serial)
Balanced general-purpose use	All dimensions	RS(12,4)

Main Conclusions

01

Platform Validation

Successfully delivered a working FPGA benchmarking platform for ECC evaluation under cluster faults.

02

Fault Tolerance

C-RRNS-MLD shows strongest burst-fault tolerance in explored design space.

03

Performance

2NRM-RRNS Parallel offers lowest decode latency with high area and power cost.

04

Storage Efficiency

2NRM-RRNS provides best storage efficiency for overhead-sensitive systems.

05

Verification

Close agreement between FPGA and MATLAB results confirms correctness and reliability.

Contributions and Future Work

Contributions

FPGA Evaluation Platform

Reusable platform with unified wrappers for standardized ECC testing

Fault Injection Engine

Low-resource probabilistic fault injection with controlled reproducibility

Trade-off Quantification

First hardware-level quantification of parallel vs serial 2NRM-RRNS trade-off

Future Work

Expand Evaluation Scope

Evaluate wider data words and additional ECC variants

Enhance Randomization

Reduce LFSR correlation with stronger random sources

ASIC Migration

Migrate framework toward ASIC or real hybrid memory devices

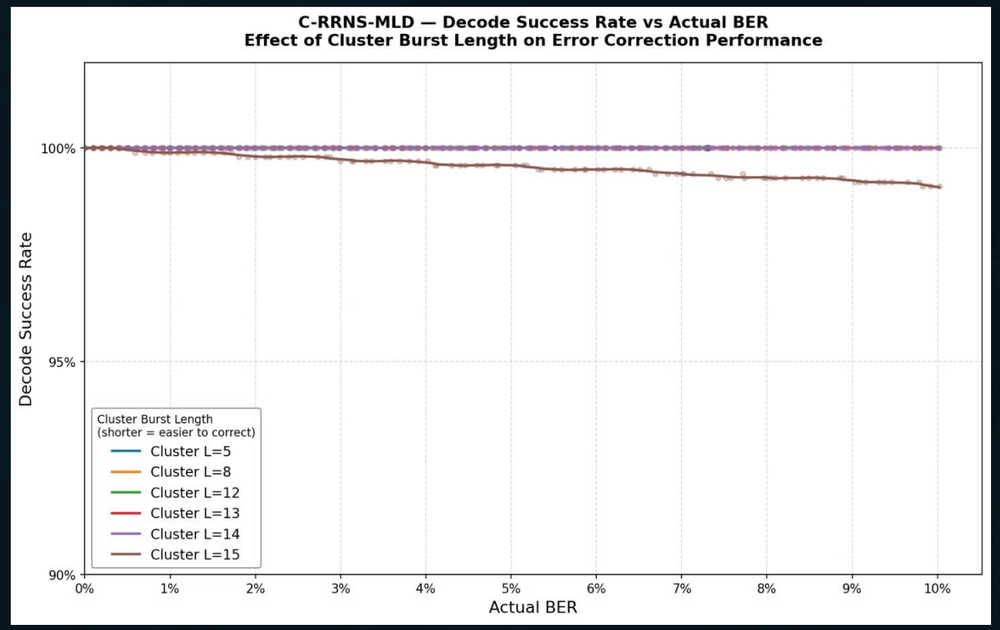
Appendix A: Fault Injection Algorithm

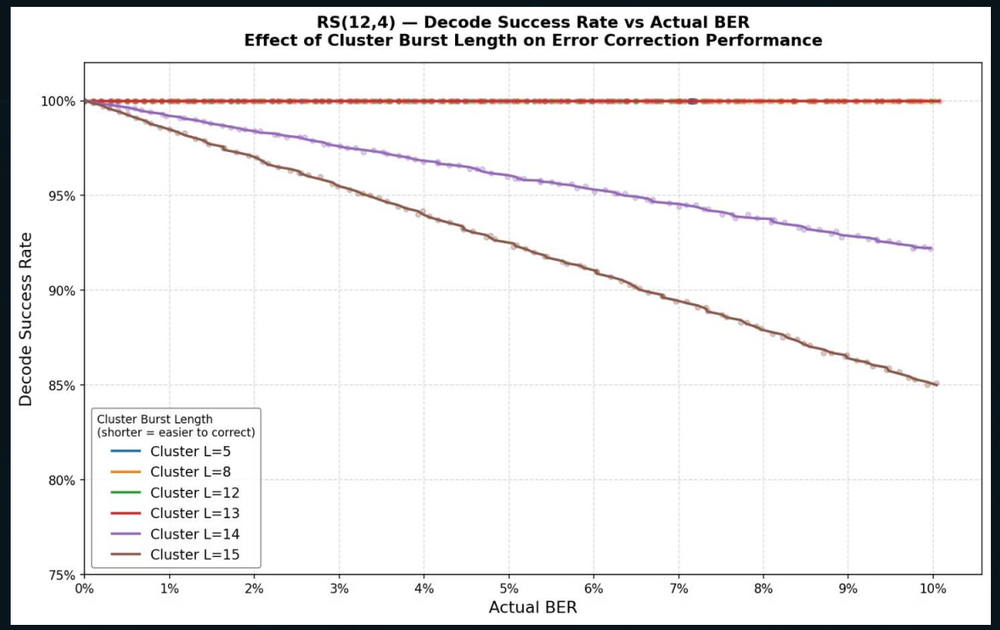
Probabilistic Injection Procedure

1. **Generate Encoded Codeword**
   Create encoded data word for testing
2. **Lock Random Seed**
   Lock one random seed for full BER sweep
3. **Compute Threshold**
   For each BER point, compute injection threshold
4. **Generate Errors**
   Use 32-bit Galois LFSR to decide error injection
5. **Apply Mode**
   Test valid bits independently or apply ROM-based burst mask
6. **Decode & Verify**
   Decode and compare with original data to record results

Controlled Evaluation: This appendix demonstrates the fault injector is not a simple random flip unit, but a **controlled and reproducible hardware evaluation algorithm** with deterministic behavior.

Appendix B: Burst-Length Impact



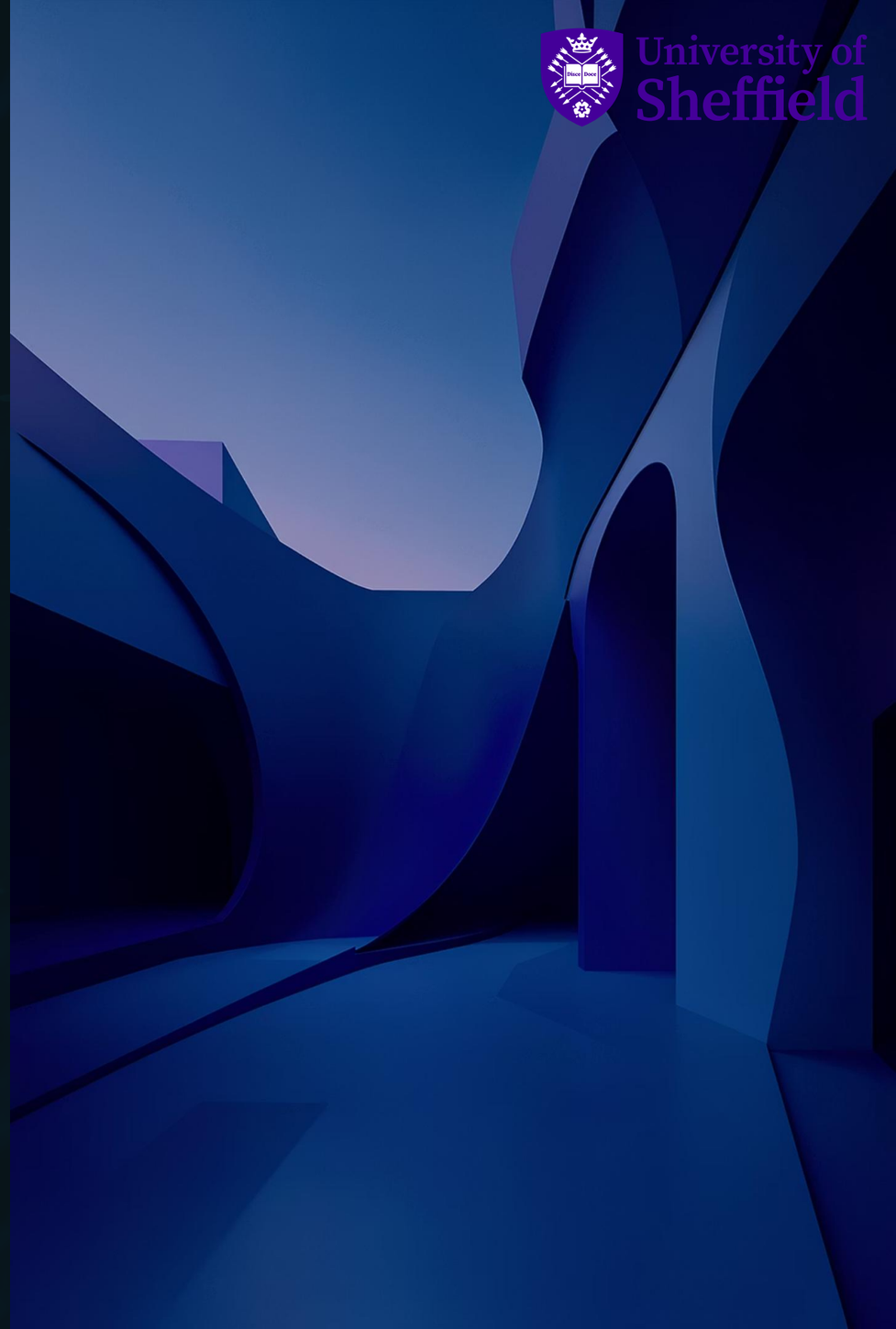


Backup Results

These curves support the final claim that **C-RRNS-MLD** provides the best extreme cluster-fault tolerance, with **RS** as the strongest balanced alternative.

Thanks

Yuqi Guo

Sheffield University

May , 2026